# Multi-Modal Machine Learning Approach for Threat Detection in WhatsApp and Telegram: Spam Links, Document Based Malware, Chat Behavior, and Sensitive Content Analysis

Detection Of Malware- Infected Documents Using Sandbox And Ml Base Analysis In Whatsapp & Telegram

25-26J-065

Final Project Proposal Report

B.Sc. (Hons) in Information Technology Specializing in Cyber Security Sri Lanka Institute of Information Technology Sri Lanka

September 2025

## Declaration

I declare that this is my own work, and this proposal does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously publish or written by another person expect where the acknowledgement is made in the text.

| Name | Student ID | Signature |
|---|---|---|
| Hitihaminelage K.I.S. | IT22561220 | |

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

…….. Date:…..27/08/2025

Signature of the supervisor:
(Mr. Deemantha Siriwardana)

…….. Date:…..27/08/2025

Signature of the Co-supervisor:
(Miss. Ayesha Wijesooriya)

## Abstract

This project proposal focuses on improving the security of messaging platforms by enabling users to check for malware-infected documents through sandbox-based behavior analysis. The primary objective is to design a secure and lightweight detection mechanism where users can manually upload or forward documents such as PDFs, DOCs, PPTX and EXCEL received via WhatsApp into the application for verification. Once uploaded, the document is executed in a sandbox environment that simulates real-world behavior in a controlled setting. The system then analyzes behavioral indicators such as file modifications, registry access, API calls, and network activity to determine if the file is malicious. If the document is found to be safe, the application will generate an alert notifying the user that it can be opened without risk. Conversely, if malicious activity is detected, the system will warn the user not to open the file. For cases where the file must still be accessed despite potential risks, the document can be safely opened and interacted with inside the sandbox without compromising the user's device. This research addresses limitations in traditional antivirus methods, which often rely on static scans and fail to detect hidden threats such as macro viruses, embedded scripts, and polymorphic malware. The methodology includes sandbox-based dynamic analysis, machine learning classification of malicious behaviors, and integration with a simple mobile interface for user interaction. By combining dynamic behavior monitoring with user-controlled file submission, the proposed solution aims to provide a proactive, accurate, and user-friendly framework for securing document sharing in messaging platforms.

## Table of Contents


I. **INTRODUCTION** ..... 7

I.1 Background and Literature Review ..... 7

I.2 Research Gap ..... 9

I.3 Research Problem ..... 11

II. **OBJECTIVES** ..... 12

II.1 Main Objective ..... 12

II.2 Sub-Objectives ..... 12

III. **METHODOLOGY** ..... 13

III.1 Requirement Gathering ..... 15

III.2 Past Research Analysis ..... 16

III.2.1 Literature Review of Key Studies: ..... 16

III.2.2 Identification of Gaps ..... 17

III.3 Feasibility Study ..... 18

III.3.1 Technical Feasibility ..... 18

III.3.1.1 Tools and Technology ..... 18

III.3.2.1 Algorithm and Architectures ..... 19

III.3.3.1 Techniques ..... 20

IV **PROJECT REQUIREMENTS** ..... 21
IV.1 Functional Requirements ..... 21
IV.2 Non-Functional Requirements ..... 22

V. **WORK BREAKDOWN AND GRANT CHART** ..... 23

VI. **COMMERCIALIZATION** ..... 25

VII. **BUDGET AND BUDGET JUSTIFICATION** ..... 27

VIII. **SUMMARY** ..... 28

IX. **REFERENCES** ..... 29

## List of Abbreviations

| | | |
|---|---|---|
| ML | - | Machine Learning |
| API | - | Application Programming Interface |
| PDF | - | Portable Document Format |
| DOC | - | Microsoft Word Document |
| UI | - | User Interface |
| VM | - | Virtual Machine |
| OP | - | Operating System |
| TP | - | True Positive |
| FP | - | False Positive |
| ROC | - | Receiver Operating Characteristic |
| AUC | - | Area Under Curve |

# I. INTRODUCTION

## I.1 Background and Literature Review

The rapid growth of social media and messaging platforms such as WhatsApp and Telegram has reshaped digital communication by allowing instant information sharing among a wide range of users. Alongside these benefits, there is now greater exposure to cyber threats, particularly from malicious files delivered through shared documents. Attackers often exploit common formats such as PDFs, DOCs, PPTX and EXCELs to spread malware that can compromise devices, extract sensitive data, or launch further attacks.

Traditional antivirus systems rely mainly on static signature-based detection. While these methods can identify known threats, they are less effective against polymorphic malware, macro viruses, and files containing obfuscated code. To address these limitations, researchers have increasingly turned to dynamic and behavior-based analysis, which is better suited for uncovering hidden or evolving threats within messaging platforms.

Sandbox-based detection is a promising approach in this area. By executing suspicious files in a controlled environment, sandboxes capture runtime behaviors such as API calls, file modifications, registry changes, and network activity. These indicators provide richer evidence than static analysis and make it possible to detect zero-day and polymorphic malware. Although sandboxing has long been used in enterprise security, its integration into real-time mobile messaging platforms remains uncommon. A major challenge lies in creating sandbox systems that are lightweight and efficient enough to operate on mobile devices without affecting usability.

To explore this gap, a systematic literature review (SLR) was conducted, covering 315 papers from IEEE Xplore, ScienceDirect, Wiley, Google Scholar, and other trusted sources. Using the PRISMA framework, ten key review papers were developed, grouped into themes such as sandbox-based dynamic analysis, multi-file-type malware detection, machine learning–based classification, and mobile security. Findings confirm that while malware detection has advanced significantly, very few solutions apply sandbox-based behavior analysis directly within messaging environments.

This study addresses that limitation by proposing a sandbox-integrated mobile application that

inspects documents shared through WhatsApp and Telegram. The system allows a received file to be redirected into a sandbox before the user can open it. Inside this environment, runtime behavior is observed and features such as API calls, registry activity, and file system changes are extracted. A machine learning model then classifies the file as benign or malicious.

If the file is safe, the user is informed that it can be opened without risk. If the file is malicious, a warning is displayed, offering the user two options: either block the file or, if the content is essential, open it in a secure sandbox viewer where it cannot harm the device. This workflow ensures that children, who may be more likely to open harmful files, are protected from infection while still being able to access legitimate content when needed.

In summary, this research contributes a novel use of sandbox-based behavior analysis in messaging platforms to detect document-based malware. Unlike static antivirus scans, this method simulates real file execution, exposing threats such as macro viruses, embedded scripts, and polymorphic malware that often evade traditional detection. By embedding this capability within a child-safety–oriented mobile application, the work provides a practical, accurate, and user-friendly method for protecting WhatsApp and Telegram users from document-borne cyber threats in real time

## I.2 Research Gap

Although research in malware detection has progressed in recent years, significant limitations remain when adapting these methods to messaging platforms such as WhatsApp and Telegram.

Research [1] applies static and dynamic analysis to detect malicious PDF and Office documents using ensemble learning. While it achieves reasonable detection rates, it lacks sandbox-based behavioral monitoring, which reduces its effectiveness against polymorphic malware and obfuscated code.

Research [2] focuses on classifying malware in Word documents using machine learning. Although ML improves accuracy, the study is restricted to Word file types and does not extend to PDFs, PPTX, or Excel documents. It also omits sandbox-based analysis, which limits its ability to detect hidden threats such as macro viruses or embedded scripts.

Research [3] introduces sandbox-based detection for implicitly malicious Word documents. While sandboxing strengthens detection, the proposed solution is computationally heavy and designed for enterprise systems, making it unsuitable for lightweight, mobile-friendly integration into real-time messaging environments.

Taken together, these studies highlight a clear gap: there is no efficient, mobile-oriented malware detection system that combines static and dynamic analysis, sandbox-based behavioral detection, and machine learning across multiple document formats. Existing methods are either limited to specific file types, omit sandboxing altogether, or are too resource-intensive for mobile deployment.

The proposed solution addresses this need by developing a lightweight sandbox-based malware detection system tailored for WhatsApp and Telegram. Documents such as PDFs, DOCs, PPTX and Excel are executed in a secure sandbox environment, where runtime behaviors—including API calls, registry changes, file modifications, and network communication—are captured. These behavioral features are then classified using machine learning to determine whether a file is benign or malicious. If identified as malicious, the system issues an alert and allows the file to be opened only in a secure sandbox viewer. This ensures accurate detection of advanced threats while remaining efficient and practical for real-world mobile messaging use.

*Table I-1 - Research Gap in tabular view*

| Research Paper | Static Analysis | Dynamic Analysis | ML-Based Malware Classification | Sandbox-Based Detection | Cover all the file type (pdf , doc, pptx, excel) | Lightweight Mobile-Friendly |
|---|---|---|---|---|---|---|
| **Research [1]** | Implemented | Implemented | Implemented | Not Implemented | Not Implemented | Not Implemented |
| **Research [2]** | Not Implemented | Implemented | Implemented | Implemented | Not Implemented | Not Implemented |
| **Research [3]** | Implemented | Implemented | Implemented | Not Implemented | Not Implemented | Not Implemented |
| **Proposed Solution** | Implemented | Implemented | Implemented | Implemented | Implemented | Implemented |

The table highlights how earlier research has addressed certain aspects of malware detection but left major gaps unresolved.

Research [1] applies static and dynamic analysis with machine learning for PDF and Office documents. However, it does not use sandbox-based detection, supports only limited file types, and is not optimized for mobile platforms.

Research [2] demonstrates machine learning–based malware classification for Word documents and incorporates sandboxing, but it is restricted to Word files only and lacks mobile-friendly integration.

Research [3] combines static and dynamic analysis with ML for Word documents, but it omits sandbox detection, covers limited file formats, and is too resource-intensive for lightweight messaging use.

By comparison, the proposed solution integrates all key elements: static and dynamic analysis, ML-based classification, sandbox-based detection, broad file-type coverage (PDF, DOC, PPTX, Excel), and a lightweight mobile-friendly design. This combination makes it particularly suitable for real-time deployment in WhatsApp and Telegram, ensuring accurate detection of document-based malware while remaining efficient and practical for everyday mobile use.

## I.3 Research Problem

The rise of messaging platforms such as WhatsApp and Telegram has created new opportunities for communication, particularly during the COVID-19 pandemic when children relied heavily on these platforms for learning and interaction. However, this increased usage has also exposed them to cybersecurity threats, one of the most critical being document-based malware. Attackers frequently embed malicious code in familiar file formats such as PDFs, DOCs, PPTX and EXCEL. Once opened, these files can execute hidden instructions, resulting in file system changes, registry modifications, or unauthorized network activity.

Conventional antivirus software relies mainly on static detection methods, which are limited in scope. These methods struggle to identify advanced threats such as polymorphic malware, obfuscated scripts, and macro viruses embedded in documents. While sandbox-based analysis has proven effective in enterprise environments, existing systems are resource-heavy and unsuitable for mobile devices commonly used by children. As a result, young users remain vulnerable to malicious documents shared through messaging platforms.

The absence of lightweight, child-friendly security solutions creates a serious gap. Parents and teachers cannot constantly monitor children's online activities, and children often lack the technical knowledge to recognize harmful files. This lack of protection increases the risk of exposure to malware, which may compromise personal devices, steal sensitive data, or enable further exploitation.

To address this problem, there is a growing demand for a parental-control–oriented mobile application that can analyze shared documents before children open them. By allowing users to upload or forward files into a sandbox environment, the system can simulate document execution, extract behavioral features, and classify files as safe or malicious. Alerts will then guide users: safe files can be opened, while malicious files trigger warnings to prevent harm.

The proposed research therefore seeks to bridge this gap by integrating sandbox-based behavioral analysis with machine learning into a lightweight mobile framework. Unlike existing approaches, it directly targets child safety in messaging platforms, empowering parents with a reliable control mechanism and ensuring that children are shielded from malicious documents during online communication.

# II. OBJECTIVES

## II.1 Main Objective

The primary objective of this research is to design and develop an ML and sandbox-based behavioral detection system for identifying malware-infected documents shared through messaging platforms such as WhatsApp and Telegram. The proposed system will provide a lightweight and user-focused mechanism that executes files in a controlled environment before they are opened. In doing so, it aims to strengthen child protection by supporting a parental-control–oriented mobile application that ensures safer use of messaging platforms.

## II.2 Sub-Objectives

- Review existing approaches: Study static and dynamic malware detection techniques and identify their limitations in mobile and messaging environments.

- Sandbox design and implementation: Build a secure sandbox to execute documents (PDF, DOCX, PPTX, Excel) and capture behavioral features such as file changes, registry activity, API calls, and network communication.

- Machine learning classification: Train ML models to classify files as benign or malicious using sandbox logs and additional threat intelligence datasets.

- Mobile application integration: Incorporate the detection system into a WhatsApp/Telegram mobile app with a user-friendly interface for file submission and alerts.

- Lightweight optimization: Ensure low resource usage and minimal performance impact so the system remains suitable for smartphones.

- Evaluation and validation: Measure accuracy, precision, recall, F1-score, and efficiency to confirm effectiveness for child protection and parental control in real-time messaging.

# III. METHODOLOGY

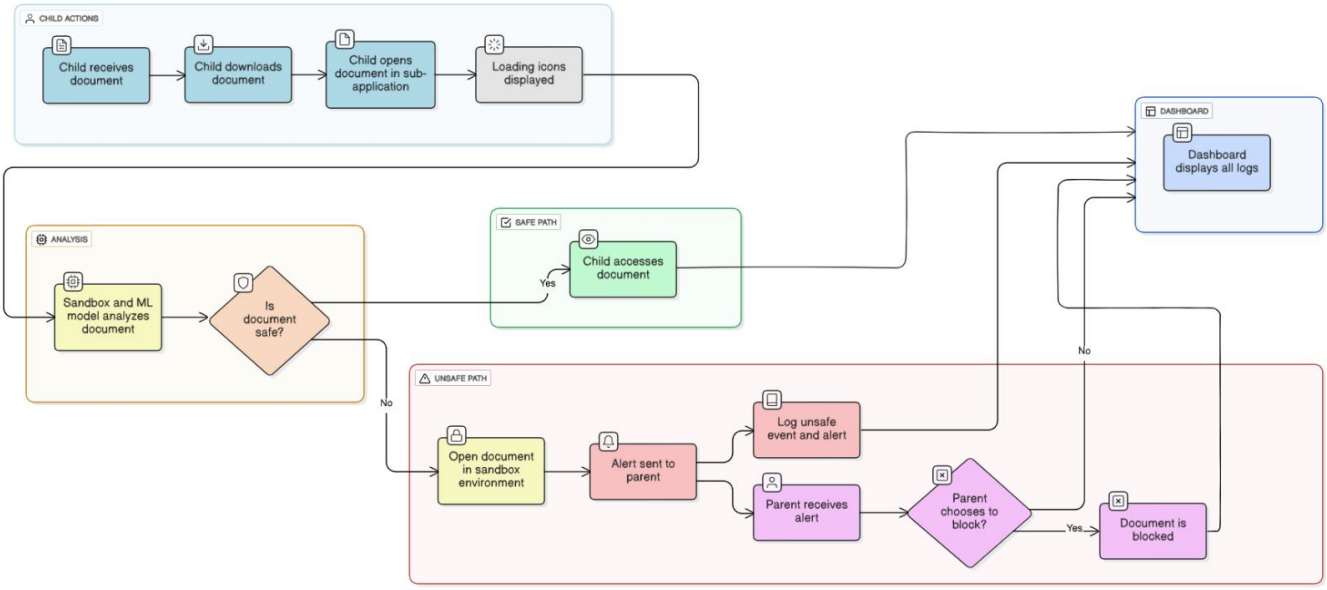


**Figure 1 - System Diagram**

The system is designed to ensure that children cannot unknowingly open malware-infected documents on messaging platforms such as WhatsApp and Telegram. Parents enable the app beforehand, so every file is automatically analyzed before access.

1. **Document Reception & Opening Attempt**
   - A child receives and downloads a document from WhatsApp/Telegram.
   - When the child tries to open it, the document is first redirected into the security sub-application.
2. **Loading & Preprocessing**
   - A temporary loading screen is displayed while the document is passed into the analysis pipeline.
3. **Sandbox & ML-Based Analysis**
   - The document is executed inside a secure sandbox.
   - Runtime behaviors are captured, such as file modifications, registry activity, API calls, and

network requests.

- These features are then classified by a machine learning model to determine whether the document is safe.

4. **Decision Phase – Safe vs Unsafe**
   - **If Safe:** The document is released to the child, who can open it normally.
   - **If Unsafe:** The document is opened only in a sandbox viewer, preventing it from harming the device.
5. **Parental Alert Mechanism**
   - An alert is sent to the parent's device.
   - The parent can review the unsafe event, shown in the dashboard logs.
   - Parents are given the choice to **block the document permanently** or allow restricted access.
6. **Blocking & Logging**
   - If the parent decides to block, the document is locked and cannot be opened by the child.
   - All unsafe events and alerts are logged in the **parental dashboard**, ensuring visibility into attempted threats.

## III.1 Requirement Gathering

The requirement gathering process combined a study of existing literature, analysis of current malware detection practices, and consultation with professionals. This ensured that the proposed solution is grounded in both academic research and practical needs.

- **Extensive Literature Review**: A systematic review of recent studies and journal articles was carried out on malware detection in documents, sandbox-based behavioral analysis, machine learning classification, and parental control applications. This review helped identify existing detection techniques, their limitations, and areas for improvement, particularly within messaging environments such as WhatsApp and Telegram.
- **Stakeholder Consultation**: Discussions and interviews were conducted with cybersecurity experts, mobile application developers, and child protection specialists. Their input provided practical insights into usability requirements, real-world malware threats, and the technical challenges of running sandbox mechanisms on mobile devices without reducing performance.
- **System Requirements Analysis**: Both functional and non-functional requirements were documented. Functional needs include sandbox integration for document execution, accurate malware classification, and real-time alerting. Non-functional needs cover performance optimization, resource efficiency, and child-safety features suitable for a parental-control– oriented mobile app. This analysis guided the design of a system that is secure, efficient, and appropriate for mobile use.
- **Use Case Scenarios**: Practical scenarios were created to validate the system design. For instance, when a child receives a suspicious PDF in WhatsApp, the file is forwarded to the security app. The app executes the file in a sandbox, analyzes its behavior, and classifies it as safe or malicious. An alert is then displayed to both the child and the parent, ensuring that harmful files are blocked while safe documents remain accessible. These scenarios confirm the system’s relevance and effectiveness in protecting children and general users from document-based malware.

## III.2 Past Research Analysis

The review of earlier studies provides important insight into how document-based malware has been addressed in different contexts of messaging and file sharing. At the same time, it shows where existing approaches fall short, especially when applied to mobile platforms such as WhatsApp and Telegram.

### III.2.1 Literature Review of Key Studies:

- Research [1]: Concentrated on static signature-based detection of malicious PDF and DOC files. While the method worked for known malware, it performed poorly against polymorphic or obfuscated threats.
- Research [2]: Used sandboxing techniques to detect malicious scripts in office documents. The study, however, was limited to desktop environments and did not consider use on mobile devices or in messaging platforms.
- Research [3]: Proposed machine learning approaches for detecting macro viruses within documents. These models, though effective in controlled datasets, lacked validation in real-world messaging environments such as WhatsApp or Telegram.
- Research [4]: Examined parental control applications designed for children's online safety. Most of these tools concentrated on web filtering and activity monitoring, with little focus on sandbox-based malware analysis for shared documents.

### III.2.2 Identification of Gaps

- Integration of sandbox-based behavioral analysis in messaging platforms: Although sandboxing is common in enterprise security, few studies have applied it directly to mobile messaging apps where document sharing is widespread.

- Multi-file malware detection: Previous research often concentrates on a single file format, such as PDFs or Word documents. There is little evidence of frameworks able to analyze multiple file types (PDFs, DOCs, PPTX and EXCEL) in one system.

- Lightweight mobile implementation: Existing sandbox systems are usually resource-intensive, making them impractical for smartphones. Very few studies address lightweight designs that can balance detection accuracy with mobile device performance.

- User-centric and child-safety context: Prior work has not given adequate attention to workflows that let users or parents decide which documents to analyze before opening. Nor has it targeted protection specifically for children using messaging platforms.

## III.3 Feasibility Study

The feasibility of this research is assessed by examining the tools, technologies, and analytical methods needed for the development and implementation of a sandbox-based malware detection system in messaging platforms. The study considers both the availability of resources and the suitability of the chosen approaches for deployment on mobile devices.

### III.3.1 Technical Feasibility

#### III.3.1.1 Tools and Technology

- **Python**: Acts as the primary programming language for implementing machine learning models, preprocessing pipelines, and integration with sandbox logs. Widely supported libraries make it ideal for behavior-based malware detection research.

- **Flutter Framework**: Used to develop a cross-platform mobile application (Android & iOS) that provides the user interface for uploading/forwarding suspicious documents. Flutter ensures lightweight performance and consistent design across both operating systems.

- **Cuckoo Sandbox / Any.Run**: Provides isolated execution environments for suspicious documents (PDFs, DOCs, PPTX, EXCEL ). These tools capture behavioral data such as API calls, file modifications, registry activity, and network traffic during execution.

- **Scikit-learn & TensorFlow/PyTorch**: Libraries to build and train machine learning classifiers for identifying malicious vs. benign files. TensorFlow/PyTorch will handle deep learning experiments if required, while Scikit-learn supports traditional algorithms such as Random Forest or SVM.

- **SQLite / Firebase**: Databases to log results, store sandbox analysis metadata, and

support real-time communication between the mobile app and the backend. Firebase also enables cloud-based updates for threat intelligence.

- **Docker**: Used to containerize the sandbox environment, ensuring safe execution of suspicious files without affecting the host system. This also allows reproducibility and scalability during testing.

#### III.3.2.1 Algorithm and Architectures

- **Sandbox-Based Dynamic Analysis**: Suspicious files are executed in a controlled sandbox, capturing runtime events such as system calls, process creation, registry changes, and network activity.
- **Machine Learning Models:**
    - Traditional ML models: Random Forest, Support Vector Machine, Logistic Regression.
    - Lightweight Neural Networks: Optimized for mobile-based deployment.
      These models classify documents as benign or malicious using sandbox-derived features.
- **Hybrid Detection Architecture:** Combines static pre-checks (macro detection, suspicious metadata, obfuscated scripts) with dynamic analysis to improve detection accuracy and reduce unnecessary sandbox executions.
- **Mobile Application Architecture:** A parental-control–oriented security app built in Flutter. The app allows users to forward suspicious files, view safety alerts, and optionally open risky documents inside a secure in-app sandbox viewer.

#### III.3.3.1 Techniques

- **Data Preprocessing**:
  - Cleaning sandbox logs and normalizing extracted features.
  - Feature engineering from behavioral indicators (e.g., frequency of API calls, abnormal network requests).

- **Sandbox Simulation**: Execution of files in isolated environments to log runtime behaviors. Multiple sandbox runs may be performed to detect evasion attempts.

- **Behavioral Feature Extraction**: Identifying patterns such as unusual registry modifications, persistent processes, or command-and-control traffic.

- **Classification & Evaluation**: ML models trained and validated using datasets of both benign and malicious files. Metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC will be used.

- **Performance Optimization**:
  - Lightweight model deployment (using TensorFlow Lite for mobile).
  - Efficient resource handling so the app runs smoothly without draining device battery or storage.

- **User Alert & Logging**: Clear visual alerts (e.g., *Safe to Open* or *Malicious File Detected*) with secure sandbox-viewing options. Logs are securely stored for parental review and future retraining of ML models.

# IV PROJECT REQUIREMENTS

## IV.1 Functional Requirements

- Document Input Handling: The system should allow users to upload or forward documents (PDFs, DOCs, PPTX and EXCELs) directly from messaging apps such as WhatsApp or Telegram into the security application for analysis.

- Sandbox Execution: Files must be executed in a secure, isolated environment where
  their runtime behavior can be observed without risking the user's device.

- Behavioral Feature Extraction: The system should record relevant behavior during execution, such as API calls, registry changes, file system actions, and network requests, for later classification.

- Malware Classification: A machine learning model must analyze the extracted behavior and classify the document as either benign or malicious.

- Alert System: After classification, the system should generate a clear result. If the file is safe, the user is notified and allowed to open it. If the file is malicious, the system should issue a warning and block the file or allow secure viewing.

- Parental Interaction (Optional): The system may support parental controls that notify
  a parent when a potentially dangerous file is detected on the child's device.

## IV.2 Non-Functional Requirements

- Accuracy: The malware detection system should offer a high level of precision and recall, ensuring that both false positives and false negatives are minimized.
- Lightweight Performance: The application must be optimized to run on mobile devices without noticeable delay or excessive use of battery, memory, or storage.
- Usability: The user interface must be simple and accessible, especially for children and non-technical users. Prompts and alerts should be clear and actionable.
- Security: The sandbox environment and app must protect user data and ensure that potentially malicious files cannot escape the analysis container.
- Compatibility: The system should support common Android versions and be capable of processing widely used document formats without compatibility issues.
- Maintainability: The architecture should support updates to machine learning models and threat data so the detection system can adapt to evolving threats.

# V. WORK BREAKDOWN AND GRANT CHART

Figure 2 – Grant chart

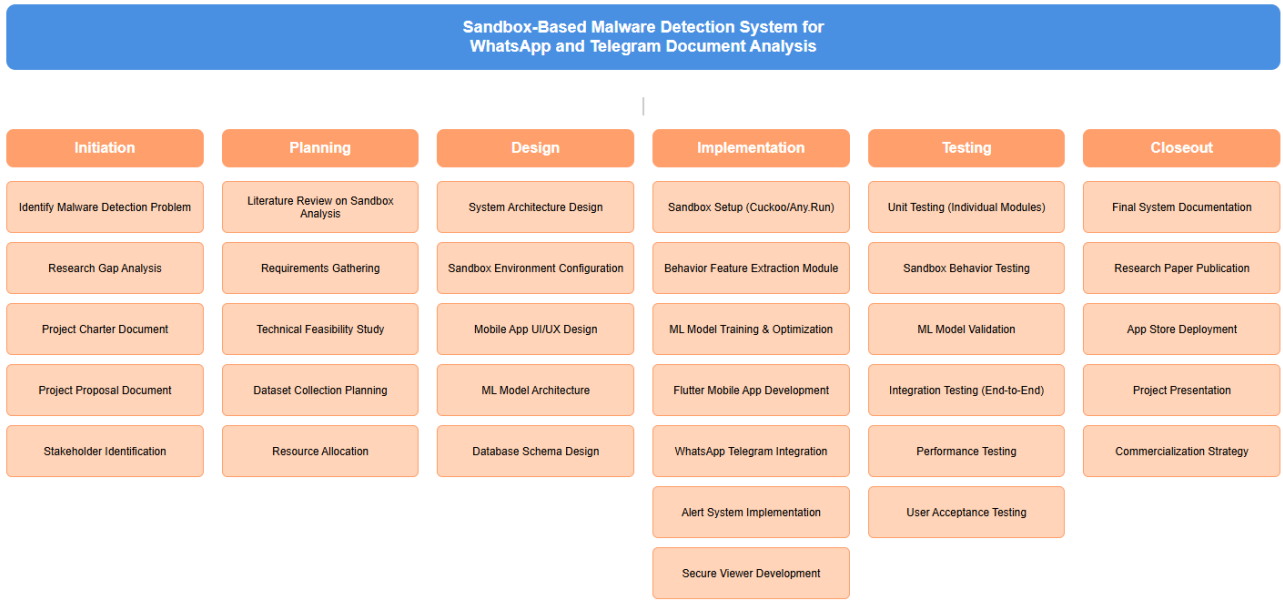


Figure 3 - Work Breakdown Chart

# VI. COMMERCIALIZATION

The proposed sandbox-based malware detection system has strong potential as a mobile security solution for real-time messaging platforms like WhatsApp and Telegram. With children and young users increasingly exposed to harmful document-based threats, this app provides parents and schools with a lightweight, reliable way to protect devices.

Unlike traditional antivirus apps, this solution is tailored to messaging environments and combines sandbox-based behavioral analysis with machine learning. It ensures safe document usage by allowing parents and children to view files securely without risking infections.

**Target Market**

- Parents of school-age children
- Schools and IT administrators
- Family security and device management tools
- Messaging platform vendors and mobile device manufacturers

**Unique Selling Proposition (USP)**

A child-focused mobile security app that detects document-based malware in real time, using behavioral sandbox analysis rather than static scanning. It provides safe viewing modes while maintaining usability for educational and communication needs.

**Revenue Model**

- **Basic ($15):** URL & Document Analysis
- **Pro ($30):** URL, Document & Chat Analysis
- **Premium ($50):** Full features – URL, Document, Chat, Media (images/videos), Parent Dashboard
- Additional revenue: licensing to schools, family security bundles, and institutional device programs

**Availability**

- Google Play Store (Android)
- Apple App Store (iOS)

**Promotion Strategy**

- Social media campaigns (Facebook, Instagram, TikTok)
- Targeted ads for parents, schools, and NGOs
- Partnerships with child-safety organizations
- Free demos and trial versions for schools

# VII. BUDGET AND BUDGET JUSTIFICATION

As the proposed research is software-oriented, no physical hardware is required. The core development involves sandbox analysis, machine learning, and mobile app integration, all of which can be carried out using open-source tools and university resources. The main expenses relate to computing infrastructure, internet access, publication, and essential research materials. A summary of the estimated costs is provided below.

*Table VII-1 - Budget Analysis*

| Category | Description | Estimated Cost (LKR) |
|---|---|---|
| Software & Tools | Python, Scikit-learn, TensorFlow Lite, Docker, Flutter (all open-source). | 0 |
| Sandbox / Cloud Testing | Occasional cloud VM usage for sandbox runs (AWS/GCP/Azure) to test documents. | 20,000 |
| Threat Intelligence APIs | Limited paid usage of services like VirusTotal / Hybrid Analysis for training & validation. | 15,000 |
| Mobile Test Devices | 2 Android devices (mid-range) for integration and sandbox behavior testing. | 60,000 |
| Dataset Handling | Storage, preprocessing, and dataset acquisition costs (benign + malicious docs). | 10,000 |
| Publication & Dissemination | Journal/conference submission fees, poster printing. | 15,000 |
| Miscellaneous | Internet, electricity, backups, contingency. | 10,000 |
| Total | | 130,000 |

This budget ensures all aspects of the research—from model development and sandbox testing to mobile app validation and dissemination—are well supported, while keeping expenses within a reasonable limit.

# VIII. SUMMARY

This project proposes a lightweight, mobile-based malware detection system designed to protect users—particularly children—from document-based threats shared through messaging platforms such as WhatsApp and Telegram. The system uses sandbox-based behavioral analysis to execute and observe files in a secure environment, capturing runtime behaviors such as file changes, registry activity, and network communication. These features are then analyzed using machine learning models to classify files as safe or malicious. The application allows users to forward suspicious files for analysis and receive alerts before opening them. In cases where a file is needed despite risk, the system offers a secure viewing option. The project includes a review of current detection methods, identification of research gaps, and validation through prototype testing. A commercialization plan targeting parents, schools, and child-safety organizations is also outlined to support broader adoption.

# IX. REFERENCES


[1] Vaishanav, L., Chauhan, S., Vaishanav, H., Sankhla, M. S., & Kumar, R. (2017). Behavioural Analysis of Android Malware using Machine Learning. International Journal Of Engineering And Computer Science, 6(5), 21378-21389. DOI: 10.18535/ijecs/v6i4.32.

[2] Aafer, Y., Du, W., & Yin, H. (2013). DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android. In SecureComm, pp. 1-18.

[3] Shakya, S., & Dave, M. (2022). Analysis, Detection, and Classification of Android Malware using System Calls. arXiv preprint arXiv:2208.06130.

[4] Riasat, R., et al. (2017). Machine Learning Approach for Malware Detection by Using APKs. In 2017 2nd International Conference on Computer, Network Security and Communication Engineering (CNSCE 2017), pp. 97-107. ISBN: 978-1-60595-439-4.

[5] Kaya, Y., Chen, Y., Botacin, M., Saha, S., Pierazzi, F., Cavallaro, L., Wagner, D., & Dumitraş, T. (2024). ML-Based Behavioral Malware Detection Is Far From a Solved Problem. arXiv preprint arXiv:2405.06124.

[6] Palma, C., Ferreira, A., & Figueiredo, M. (2024). Explainable Machine Learning for Malware Detection on Android Applications. Information, 15(1), 25. https://doi.org/10.3390/info15010025.

[7] Alswyna, N. H., & Khan, M. M. (2022). Design and Analysis of Machine Learning Based Technique for Malware Detection in PDF Files. Security and Communication Networks, 2022, Article ID 7611741. https://doi.org/10.1155/2022/7611741.

[8] Unit 42. (2023). Leveraging a Hooking Framework to Expand Malware Detection Coverage on the Android Platform. Palo Alto Networks. https://unit42.paloaltonetworks.com/hooking-framework-in- sandbox-to-analyze-android-apk/.